Google Cloud

Learn the Google Cloud model: projects, VPC, Compute Engine, Cloud Storage, GKE, Cloud Run, data and messaging services, operations and cost, with the gcloud CLI.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Google Cloud: organization, projects and billing (12 min)
2. gcloud CLI: configurations and automation (12 min)
3. VPC networks, subnets and firewall rules (13 min)
4. Compute Engine, images and managed instance groups (13 min)
5. Cloud Storage and persistent disks (12 min)
6. GKE: Autopilot and Standard clusters (13 min)
7. Cloud Run and Cloud Functions (12 min)
8. Cloud SQL, Spanner, BigQuery and Firestore (13 min)
9. Pub/Sub, Eventarc and Cloud Tasks (12 min)
10. Observability: Cloud Logging, Monitoring and SLOs (13 min)
11. Terraform on Google Cloud (12 min)
12. Security, cost and debugging on Google Cloud (14 min)
Cheat sheet

Lesson 1. Google Cloud: organization, projects and billing

12 minutes. Learn how Google Cloud organises resources into organization, folders and projects, and how IAM, billing accounts and budgets fit on top.

Before creating any server, learn the tree that holds it. Google Cloud's hierarchy decides who can do what, where policies apply and who receives the bill.

What you will learn

- Organization, folders and projects
- Project id, name and number
- IAM inheritance and billing accounts
- Budgets and labels

The hierarchy

An **organization** (tied to a Cloud Identity or Workspace domain) contains **folders** (for teams or environments) that contain **projects**. A project holds resources: virtual machines, buckets, databases. IAM policies and **organization policies** set higher up are **inherited** by everything below.

Level	Typical use
Organization	Company-wide policies and admins
Folder	Team, business unit or environment
Project	One application in one environment
Resource	A VM, bucket, topic

A project has a **name** (editable), an **id** (globally unique, permanent) and a **number**. Use the id in commands and code.

Billing

A **billing account** pays for projects linked to it. Set **budgets** with alerts at 50, 90 and 100 percent, and export billing data to BigQuery for analysis. Use **labels** (`env=prod`, `team=payments`) to split costs.

```
$ gcloud projects create acme-pay-prod --folder=123456789
$ gcloud billing projects link acme-pay-prod --billing-account=ABCDEF-123456-7890AB
$ gcloud services enable compute.googleapis.com --project=acme-pay-prod
```

APIs are **off by default** and enabled per project.

Diagram: The Google Cloud hierarchy

Part	What it does	What goes wrong
Organization	Top of the tree; company policies live here.	A policy set here applies everywhere.
Folders	Group projects by team or environment.	Deep folder trees get hard to manage.
Dev project	Test resources with relaxed access.	Shared with prod it risks production data.
Prod project	Production resources with strict access.	Broad roles here widen the blast radius.
Billing account	Pays for linked projects.	No budget means surprise bills.

Flow: Organization to Folders / Folders to Dev project / Folders to Prod project / Billing account to Prod project

Incident: Cleanup in the wrong place

Incident: *A script meant for dev deleted production disks because both lived in one project.*

```
$ gcloud compute disks delete prod-db-disk --project acme-app
Deleted [prod-db-disk]
```

MARGA by EknathaLabs marga.eknathalabs.com

```
project acme-app contains dev, test and prod resources
```

One project mixed all environments. The team created a project per environment, gave the cleanup tool permissions only in dev, and set budgets and alerts per project.

Common mistakes

- One project for every environment
- Primitive roles such as Owner given widely
- No budget alert
- Using project names instead of ids in scripts

Check yourself

What is the main boundary for permissions and billing?

Answer: The project

How do IAM policies flow?

Answer: Down the hierarchy

Are APIs enabled by default?

Answer: No, per project

What are labels for?

Answer: Cost split and organisation

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. gcloud CLI: configurations and automation

12 minutes. Use the gcloud CLI with named configurations, service account impersonation, output formats and filters, and write safe scripts.

Almost everything you do in the console can be scripted. A few habits around configurations, credentials and output formatting keep your scripts safe and your commands readable.

What you will learn

- Install, login and configurations
- Credentials for people and for code
- Formats, filters and queries
- Safe scripting habits

Setup

```
$ gcloud auth login
$ gcloud config configurations create prod
$ gcloud config set project acme-pay-prod
$ gcloud config set compute/region europe-west1
$ gcloud config configurations list
NAME  IS_ACTIVE  ACCOUNT           PROJECT
dev   False      me@acme.example   acme-pay-dev
prod  True       me@acme.example   acme-pay-prod
$ gcloud config configurations activate dev
```

A **configuration** stores account, project, region and zone. Keep one per project and make the active one visible in your prompt. Cloud Shell in the browser has the CLI ready.

Credentials

- `gcloud auth login` signs in **you** for CLI commands.
- `gcloud auth application-default login` creates **Application Default Credentials** that client libraries and Terraform use locally.
- On Google Cloud, code uses the **attached service account** with no key file.
- Prefer **impersonation**:
  `--impersonate-service-account=deployer@acme-pay-prod.iam.gserviceaccount.com` runs a command as that account without downloading a key.

Output

```
$ gcloud compute instances list --filter="labels.env=prod AND status=RUNNING" \
    --format="table(name,zone.basename(),networkInterfaces[0].networkIP)"
$ gcloud compute instances describe web-1 --zone europe-west1-b --format="value(status)"
$ gcloud run services list --format=json | jq '.[].metadata.name'
```

Safe scripts

Pass `--project` and `--quiet` explicitly, use `set -euo pipefail`, run a list command first and review it before delete, and prefer Terraform for anything long-lived.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: Where gcloud gets its identity

Part	What it does	What goes wrong
You	Sign in with gcloud auth login.	A shared login hides who did what.
Configuration	Holds account, project and region.	A forgotten active project targets the wrong place.
Impersonation	Run as a service account without a key file.	Too powerful a service account negates the safety.
Google Cloud APIs	Receive the call and check IAM.	Disabled APIs reject the call.
Application default credentials	Used by libraries and Terraform locally.	A downloaded key file can leak.

Flow: You to Configuration / Configuration to Impersonation / Impersonation to Google Cloud APIs / Application default credentials to Google Cloud APIs

Incident: Deleted in the wrong project

Incident: *A delete ran against production because an old configuration was still active.*

```
$ gcloud config list
[core] project = acme-pay-prod
$ gcloud compute instances delete batch-1 --zone europe-west1-b
Deleted [batch-1]
```

The engineer believed the CLI pointed at dev. The team added the project to the shell prompt, put --project in scripts and used one configuration per environment.

Common mistakes

- Relying on whichever project happens to be active
- Downloading service account key files
- Parsing human text output instead of --format
- Scripts without a dry run or list step

Check yourself

What does a gcloud configuration hold?

Answer: Account, project and region settings

What lets code on a VM authenticate without a key file?

Answer: The attached service account

Which flag formats output as a table or value?

Answer: --format

What is safer than a downloaded key?

Answer: Service account impersonation

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. VPC networks, subnets and firewall rules

13 minutes. Design Google Cloud VPC networks with global VPCs, regional subnets, firewall rules, Cloud NAT and private access to Google services.

Networking in Google Cloud differs from other clouds in one big way: a VPC spans all regions. Learning that, and how firewall rules attach to instances, gives you a clean and safe design.

What you will learn

- Global VPCs and regional subnets
- Firewall rules and priorities
- Cloud NAT and private access
- Shared VPC and peering

VPC and subnets

A **VPC network** is global; each **subnet** belongs to one region and has an IP range. Use **custom mode** so you pick the ranges rather than accepting the auto-mode defaults, and plan non-overlapping ranges for peering and on-premises links.

```
$ gcloud compute networks create prod-vpc --subnet-mode=custom
$ gcloud compute networks subnets create app-eu --network=prod-vpc \
    --region=europe-west1 --range=10.10.0.0/20 --enable-private-ip-google-access
```

Firewall rules

Ingress is denied and egress allowed by default (implied rules). Add rules with a **priority** (lower number wins), direction, protocol, ports, source and **target** (tags or service accounts).

```
$ gcloud compute firewall-rules create allow-lb-to-web --network=prod-vpc \
    --direction=INGRESS --allow=tcp:8080 \
    --source-ranges=130.211.0.0/22,35.191.0.0/16 \
    --target-service-accounts=web@acme-pay-prod.iam.gserviceaccount.com
$ gcloud compute ssh web-1 --tunnel-through-iap
```

Prefer **service accounts** as targets because tags can be set by anyone with edit rights on the instance. **IAP** tunnelling gives administrators SSH without opening port 22 to the internet.

Reaching out

VMs without public IPs reach the internet through **Cloud NAT** (via a Cloud Router) and reach Google APIs through **Private Google Access**. **Shared VPC** lets a host project own the network while service projects deploy into it. **VPC peering** connects two VPCs privately; it is not transitive.

Diagram: A private Google Cloud network

Part	What it does	What goes wrong
Users	Reach the load balancer on a public address.	An open admin port invites attacks.
Load balancer	Terminates traffic and forwards to backends.	Missing health-check ranges in the firewall mark backends unhealthy.
Private VMs	No public IP; rules by service account.	A broad allow rule defeats the isolation.
Cloud NAT	Outbound internet without public IPs.	Port exhaustion breaks many outbound connections.
IAP tunnel	Admin SSH without a public port.	Granting IAP tunnel access too widely replaces one open door with another.

Flow: Users to Load balancer / Load balancer to Private VMs / Private VMs to Cloud NAT / IAP tunnel to Private VMs

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: Backends unhealthy

Incident: *A new firewall rule set blocked the load balancer's health checks.*

```
$ gcloud compute backend-services get-health web --global
web-1  healthState: UNHEALTHY
firewall: no rule allowing 130.211.0.0/22 and 35.191.0.0/16 on tcp:8080
```

The load balancer probes come from Google ranges. Allowing those ranges to the service port brought backends back.

Common mistakes

- Using the default network in production
- Firewall rules based on tags that developers can edit
- Overlapping IP ranges between VPCs
- Forgetting health-check source ranges

Check yourself

What is the scope of a Google Cloud VPC?

Answer: Global

What are the default firewall behaviours?

Answer: Deny ingress, allow egress

How do private VMs reach the internet?

Answer: Cloud NAT

Which gives admin SSH without an open port 22?

Answer: IAP tunnelling

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Compute Engine, images and managed instance groups

13 minutes. Create Compute Engine VMs, choose machine types and images, and run them in managed instance groups with autoscaling, health checks and rolling updates.

A VM you log into and fix by hand is a pet. Google Cloud gives you the tools to run herds: templates, groups and health checks that rebuild failed machines for you.

What you will learn

- Machine types, images and disks
- Instance templates and managed instance groups
- Autohealing, autoscaling and rolling updates
- Spot VMs and cost control

Creating a VM

```
$ gcloud compute instances create web-1 --zone=europe-west1-b \
    --machine-type=e2-standard-2 --image-family=debian-12 --image-project=debian-cloud \
    --subnet=app-eu --no-address --service-account=web@acme-pay-prod.iam.gserviceaccount.com \
    --metadata=enable-oslogin=TRUE
```

Pick a machine family for the workload (E2 or N2 general, C for compute, M for memory, A or G for GPUs) or a custom size. Use **OS Login** to tie SSH access to IAM instead of copied keys.

Instance templates and groups

An **instance template** records machine type, image, network and startup script. A **managed instance group (MIG)** keeps N copies of it running.

```
$ gcloud compute instance-templates create web-v2 --machine-type=e2-standard-2 \
    --image-family=web-golden --subnet=app-eu --no-address
$ gcloud compute instance-groups managed create web --region=europe-west1 \
    --template=web-v2 --size=3
$ gcloud compute instance-groups managed set-autoscaling web --region=europe-west1 \
    --max-num-replicas=10 --target-cpu-utilization=0.6
```

A **regional** MIG spreads instances across zones. **Autohealing** recreates instances that fail an application health check. **Rolling updates** replace instances gradually (`max-surge`, `max-unavailable`) when you point the group at a new template.

Cost

Spot VMs cost far less but can be stopped at any moment, so use them for batch and stateless work. Rightsizing recommendations and committed use discounts reduce steady cost.

Diagram: A self-healing group of VMs

Part	What it does	What goes wrong
Golden image	The built operating system and app.	A manual image drifts from the code.
Instance template	Machine type, image, network and scripts.	Templates cannot be edited; make a new one.
Regional MIG	Keeps the wanted number of instances in several zones.	A zone with no quota leaves it short.
Health check	Tests the app; failing VMs are recreated.	A check that is too strict causes a restart loop.
Autoscaler	Adds or removes instances based on load.	Slow startup lets a spike pass unserved.

Flow: Golden image to Instance template / Instance template to Regional MIG / Regional MIG to Health check / Autoscaler to Regional MIG

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: Restart loop

> **Incident:** *Autohealing kept recreating instances because the health check timed out before the app started.*

```
$ gcloud compute instance-groups managed list-instances web --region=europe-west1
web-a1  RECREATING  UNHEALTHY
health check: initial delay 30s; app ready after 90s
```

The check started before the app was ready and declared the VM unhealthy. Raising the initial delay above the startup time stopped the loop.

Common mistakes

- Hand-built VMs with no template
- Zonal groups for services that must survive a zone failure
- Health checks with no initial delay for slow starts
- Spot VMs for stateful services

Check yourself

What keeps N identical VMs running?

Answer: A managed instance group

What does autohealing do?

Answer: Recreates instances that fail health checks

Why use a regional MIG?

Answer: To survive a zone problem

What is a Spot VM best for?

Answer: Interruptible batch work

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Cloud Storage and persistent disks

12 minutes. Use Cloud Storage buckets with classes, lifecycle rules, versioning and uniform access, and choose between Persistent Disk, Hyperdisk and Filestore.

Storage on Google Cloud comes in object, block and file forms. Most beginners need only the first two and the habits that keep data safe and cheap.

What you will learn

- Buckets, classes and locations
- Access control and signed URLs
- Lifecycle, versioning and retention
- Disks, snapshots and file storage

Buckets

```
$ gcloud storage buckets create gs://acme-exports --location=europe-west1 \
    --uniform-bucket-level-access --public-access-prevention
$ gcloud storage cp report.csv gs://acme-exports/2026/
$ gcloud storage ls --long gs://acme-exports/2026/
```

A bucket has a **location** (region, dual-region or multi-region) and a default **storage class**: Standard for hot data, Nearline (30 days), Coldline (90 days), Archive (365 days) for rarely read data. Colder classes cost less to keep and more to read, and have minimum storage durations.

Access

Use **uniform bucket-level access** so IAM alone controls access (no per-object ACLs), turn on **public access prevention**, and give service accounts the narrowest role (`roles/storage.objectViewer`). For temporary sharing, create a **signed URL**.

Lifecycle and protection

```
{"rule": [
  {"action": {"type": "SetStorageClass", "storageClass": "NEARLINE"}, "condition": {"age": 30}},
  {"action": {"type": "Delete"}, "condition": {"age": 365}}
]}
```

Object versioning keeps previous versions after overwrite or delete; a **retention policy** prevents deletion for a period. Lifecycle rules move or delete objects automatically.

Disks and files

A **Persistent Disk** (balanced, SSD) attaches to VMs and survives instance deletion if configured that way. Take scheduled **snapshots** and test restoring them. **Hyperdisk** gives tunable performance. **Filestore** offers shared NFS.

Diagram: Where data lives

Part	What it does	What goes wrong
Application	Reads and writes through IAM roles.	Overly broad roles expose all buckets.
Bucket	Objects with a class and a location.	Public access exposes data to the world.
Lifecycle rules	Move to colder classes or delete by age.	Deleting too soon loses needed data.
Versioning	Keeps older versions after change.	Unbounded versions raise cost.
Persistent Disk	Block storage for one VM with snapshots.	No snapshot schedule means no restore point.

Flow: Application to Bucket / Bucket to Lifecycle rules / Bucket to Versioning / Application to Persistent Disk

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The public bucket

Incident: *A bucket was made public for a quick share and customer exports stayed exposed.*

```
$ gcloud storage buckets get-iam-policy gs://acme-exports
members: allUsers  role: roles/storage.objectViewer
audit: 14,201 external reads
```

A one-off share became a permanent exposure. The team removed allUsers, enabled public access prevention at the organization, and used signed URLs with a short expiry for sharing.

Common mistakes

- Granting allUsers or allAuthenticatedUsers
- Archive class for data read often
- No lifecycle rules, so old data piles up
- Disks without snapshot schedules

Check yourself

What does public access prevention do?

Answer: Blocks making a bucket public

Which class suits data read once a year?

Answer: Archive or Coldline

How do you share a file temporarily?

Answer: A signed URL

What protects against accidental overwrite?

Answer: Object versioning

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. GKE: Autopilot and Standard clusters

13 minutes. Compare GKE Autopilot and Standard modes, create a cluster, use Workload Identity, node pools and release channels, and avoid common cost and upgrade traps.

GKE removes the hardest part of Kubernetes, running the control plane, and offers two levels of further help. Choosing the right mode and securing identity early prevents most surprises.

What you will learn

- Autopilot and Standard modes
- Regional clusters, release channels and upgrades
- Workload Identity and private clusters
- Ingress, Gateway and cost habits

Two modes

	Autopilot	Standard
Nodes	Managed by Google	You manage node pools
Billing	Pod resource requests	Node VMs
Privileged pods, custom kernel	Limited	Allowed
Hardened defaults	Yes	You configure

Create a cluster

```
$ gcloud container clusters create-auto prod --region=europe-west1 --release-channel=regular
$ gcloud container clusters get-credentials prod --region=europe-west1
$ kubectl get nodes
```

Regional clusters replicate the control plane across zones. A **release channel** (rapid, regular, stable) gets automatic upgrades; set **maintenance windows** and **exclusions** for busy periods.

Identity

Workload Identity Federation for GKE lets a Kubernetes service account act as a Google service account, so pods get short-lived tokens without key files.

```
$ kubectl annotate serviceaccount web -n shop \
    iam.gke.io/gcp-service-account=web@acme-pay-prod.iam.gserviceaccount.com
```

Use **private clusters** (nodes without public IPs) and authorised networks for the API endpoint.

Traffic and cost

Expose services through the **Gateway API** or Ingress to a Google load balancer. In Autopilot, set accurate resource **requests**, since they decide the bill. In Standard, use cluster autoscaler, Spot node pools for tolerant workloads and bin packing to avoid idle nodes.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A GKE cluster

Part	What it does	What goes wrong
kubectl and CI	Talks to the Kubernetes API.	A public API with no network limits is exposed.
Managed control plane	Run by Google; regional for high availability.	Version skew blocks old APIs after upgrade.
Nodes	Autopilot-managed or your node pools.	Idle nodes cost money in Standard.
Workload Identity	Maps a Kubernetes service account to a Google one.	Without it pods share the node account.
Google load balancer	Sends external traffic to pods via Gateway.	Wrong health-check path marks pods unhealthy.

Flow: kubectl and CI to Managed control plane / Managed control plane to Nodes / Workload Identity to Nodes / Nodes to Google load balancer

Incident: Pods stuck pending

> **Incident:** *New pods stayed pending because requests exceeded what the cluster could provide in Standard mode.*

```
$ kubectl describe pod web-7d9f
Warning FailedScheduling  0/3 nodes are available: 3 Insufficient cpu.
cluster-autoscaler: node pool at maximum size 3
```

The node pool maximum was too low for the new requests. Raising the maximum and rightsizing requests let pods schedule. The team later moved the workload to Autopilot to remove node sizing.

Common mistakes

- Using node service account keys instead of Workload Identity
- Requests copied from examples, inflating the bill in Autopilot
- Disabling auto-upgrades and falling far behind
- A public API endpoint with no authorised networks

Check yourself

What does Autopilot bill by?

Answer: Pod resource requests

What does Workload Identity avoid?

Answer: Key files for pods

What is a release channel?

Answer: A stream of automatic upgrades

When choose Standard?

Answer: You need privileged pods or custom nodes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Cloud Run and Cloud Functions

12 minutes. Deploy containers to Cloud Run and event code to Cloud Run functions, with scaling to zero, revisions, traffic splitting, concurrency and secrets.

For many web services and background tasks, a container platform that charges per use is simpler than servers or Kubernetes. Cloud Run is Google Cloud's answer, and it is often the right first choice.

What you will learn

- Deploying services and revisions
- Scaling, concurrency and cold starts
- Secrets, networking and authentication
- Jobs and event-driven functions

Deploy

```
$ gcloud run deploy web --image=europe-west1-docker.pkg.dev/acme-pay-prod/app/web:1.4.2 \
    --region=europe-west1 --service-account=web@acme-pay-prod.iam.gserviceaccount.com \
    --no-allow-unauthenticated --max-instances=20 --concurrency=80 \
    --set-secrets=DB_PASSWORD=db-password:latest
$ gcloud run services update-traffic web --region=europe-west1 --to-revisions=web-00007=10,web-000 \
    06=90
```

Each deploy makes a **revision**. Traffic can split between revisions for canary tests and rolled back instantly. The container must listen on the port in `$PORT` and keep no local state.

Scaling

An instance handles many requests at once up to `--concurrency`. Cloud Run adds instances when concurrency fills and scales to **zero** when idle. `--min-instances` keeps warm instances; `--max-instances` protects databases from overload. Choose CPU always allocated for background work, or CPU only during requests to save money.

Security and networking

Services are private unless you grant `roles/run.invoker`; use `--no-allow-unauthenticated` and call with identity tokens. Read secrets from **Secret Manager**. Use **Direct VPC egress** to reach private resources.

Jobs and functions

Cloud Run jobs run containers to completion on a schedule or on demand (batch, migrations). **Cloud Run functions** run small event handlers triggered by HTTP, Pub/Sub, Cloud Storage and others, built on the same platform.

Diagram: Cloud Run request path

Part	What it does	What goes wrong
Caller	Sends a request with an identity token.	Allowing unauthenticated calls exposes the service.
Cloud Run service	Routes to a revision and scales instances.	Cold starts delay the first request.
Revisions	Versions with traffic split.	Old revisions linger if never cleaned.
Secret Manager	Supplies passwords at runtime.	Hard-coded secrets leak in images.
Database	Limited by max instances and connections.	Unlimited scale exhausts connections.

Flow: Caller to Cloud Run service / Cloud Run service to Revisions / Revisions to Database / Secret Manager to Cloud Run service

Incident: Database connection storm

Incident: *Cloud Run scaled quickly and opened more connections than the database allowed.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ gcloud run services describe web --format='value(spec.template.scaling.maxInstance)'
(none)
postgres: FATAL: remaining connection slots are reserved
```

Unlimited scaling met a limited database. The team set max instances, used a connection pooler and lowered per-instance connections.

Common mistakes

- No maximum instances in front of a database
- Storing state in container local disk
- Public services without authentication needs
- Secrets in environment variables baked into the image

Check yourself

What does a deploy create?

Answer: A new revision

What does concurrency control?

Answer: Requests handled at once per instance

How are secrets supplied?

Answer: From Secret Manager at runtime

What are Cloud Run jobs for?

Answer: Run-to-completion tasks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Cloud SQL, Spanner, BigQuery and Firestore

13 minutes. Choose between Cloud SQL, AlloyDB, Spanner, Firestore and BigQuery, and run Cloud SQL safely with high availability, backups and private connections.

Choosing the wrong database is expensive to reverse. A short map of what each Google Cloud database is good at, plus safe defaults for the common one, prevents most regret.

What you will learn

- The database map
- Cloud SQL high availability and backups
- Connecting privately
- BigQuery cost control

The map

Need	Choose
Normal relational app	Cloud SQL (MySQL, PostgreSQL, SQL Server)
Heavy PostgreSQL, analytics mix	AlloyDB
Globally consistent relational, huge scale	Spanner
Documents, mobile and web	Firestore
Very high-throughput key-value	Bigtable
Analytics over large data	BigQuery

Cloud SQL safely

```
$ gcloud sql instances create orders-db --database-version=POSTGRES_16 --tier=db-custom-4-16384 \
    --region=europe-west1 --availability-type=REGIONAL --network=prod-vpc --no-assign-ip \
    --backup-start-time=02:00 --enable-point-in-time-recovery
$ gcloud sql connect orders-db --user=app
```

High availability keeps a standby in another zone with automatic failover. **Backups plus point-in-time recovery** let you restore to a moment; practise a restore. Use **private IP**, and the **Cloud SQL Auth Proxy** or connectors for IAM-based encrypted connections. Add **read replicas** for read scaling, which are not a substitute for HA.

BigQuery

BigQuery separates storage from compute and bills by data scanned (on-demand) or by reserved capacity. **Partition** tables by date, **cluster** by common filters, select only needed columns and use a dry run.

```
$ bq query --dry_run --use_legacy_sql=false 'select user_id from logs.events where dt = "2026-11-0 \
    5"'
Query successfully validated. This query will process 3.1 GB when run.
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Picking and protecting a database

Part	What it does	What goes wrong
Application	Connects privately with IAM auth.	Public IPs with passwords invite attacks.
Cloud SQL HA	Primary plus standby in another zone.	Failover causes a brief connection drop.
Backups and PITR	Restore to a point in time.	An untested restore is only a hope.
BigQuery	Analytics on partitioned tables.	A query without a partition filter scans everything.
Spanner or Firestore	Global scale and special models.	Used where Cloud SQL suffices, they cost more.

Flow: Application to Cloud SQL HA / Cloud SQL HA to Backups and PITR / Application to Spanner or Firestore / Application to BigQuery

Incident: The expensive query

Incident: *A query with no partition filter scanned a huge table.*

```
$ bq show -j bqjob_r123
Total bytes processed: 48.2 TB
table: logs.events (not partitioned)
```

The table had no partitioning or required filter. The team partitioned by date, set require_partition_filter and added a per-user daily bytes limit.

Common mistakes

- Read replicas used as high availability
- Public database IPs
- Restores never practised
- Select * and no dry run in BigQuery

Check yourself

What does Cloud SQL high availability provide?

Answer: Automatic failover to a standby

How does BigQuery on-demand pricing work?

Answer: By bytes scanned

Which is for globally consistent relational scale?

Answer: Spanner

What does a dry run show?

Answer: Bytes a query would process

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Pub/Sub, Eventarc and Cloud Tasks

12 minutes. Use Pub/Sub topics and subscriptions, Eventarc triggers and Cloud Tasks queues to connect Google Cloud services asynchronously and handle failures.

Asynchronous messaging links the parts of a Google Cloud system without making them wait for each other. Three services cover the common cases and share the same rules about retries and duplicates.

What you will learn

- Topics, subscriptions and acknowledgements
- Dead letters and retries
- Eventarc and Cloud Scheduler
- Cloud Tasks for targeted work

Pub/Sub

```
$ gcloud pubsub topics create orders
$ gcloud pubsub subscriptions create billing --topic=orders --ack-deadline=60 \
    --dead-letter-topic=orders-dead --max-delivery-attempts=5 \
    --message-retention-duration=7d
$ gcloud pubsub topics publish orders --message='{"id":4411}' --attribute=region=eu
```

A **topic** receives messages; each **subscription** gets its own copy. **Pull** subscribers fetch; **push** subscriptions call an HTTPS endpoint such as Cloud Run. Delivery is **at-least-once**. If the subscriber does not acknowledge within the **ack deadline**, the message is redelivered. After the maximum attempts it goes to the **dead-letter topic**. Retained messages and **seek** allow replay, and **ordering keys** keep messages with one key in order.

Eventarc and Scheduler

Eventarc routes events (a file uploaded to Cloud Storage, an audit log entry, a Pub/Sub message) to Cloud Run or functions. **Cloud Scheduler** triggers a topic or URL on a cron schedule.

Cloud Tasks

Cloud Tasks queues individual HTTP tasks for a specific handler, with rate limits (`maxDispatchesPerSecond`), concurrency limits and retry settings. Choose Tasks when you want to control how fast one service is called; choose Pub/Sub when many subscribers need an event.

Diagram: Asynchronous paths on Google Cloud

Part	What it does	What goes wrong
Publisher	Sends events to a topic.	A message without an id is hard to trace.
Pub/Sub topic	Copies each message to every subscription.	A missing subscription loses the event for that consumer.
Subscription	Pull or push delivery with an ack deadline.	Short deadlines cause duplicates.
Dead-letter topic	Collects messages that failed N times.	Unwatched, it hides failures.
Eventarc	Turns Google events into calls to Cloud Run.	Wrong filters send nothing.

Flow: Publisher to Pub/Sub topic / Pub/Sub topic to Subscription / Subscription to Dead-letter topic / Pub/Sub topic to Eventarc

Incident: Redelivered again and again

Incident: *Messages were delivered repeatedly because handling took longer than the acknowledgement deadline.*

```
$ gcloud pubsub subscriptions describe billing --format='value(ackDeadlineSeconds)'
10
billing log: handler took 45s for message 8812; delivery attempt 4
```

MARGA by EknathaLabs marga.eknathalabs.com

The handler outlasted the deadline, so Pub/Sub assumed failure. The team raised the deadline, made the handler idempotent and added a dead-letter topic with an alert.

Common mistakes

- Ack deadline shorter than the handler
- No dead-letter topic
- Handlers that cannot safely run twice
- Using Pub/Sub for rate-controlled calls to one service

Check yourself

What delivery guarantee does Pub/Sub give?

Answer: At-least-once

What happens when the ack deadline passes?

Answer: The message is redelivered

When use Cloud Tasks?

Answer: To control how fast one handler is called

What is Eventarc for?

Answer: Routing Google service events to handlers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Observability: Cloud Logging, Monitoring and SLOs

13 minutes. Use Cloud Logging, Cloud Monitoring, Trace and Error Reporting to find problems, write alerting policies and define service level objectives.

The operations suite is on by default for most services, so the work is to use it well: query logs quickly, make alerts meaningful and measure reliability with an objective.

What you will learn

- Logs Explorer, sinks and log-based metrics
- Metrics, dashboards and alerting policies
- Traces, errors and uptime checks
- SLOs, error budgets and burn-rate alerts

Logs

```
$ gcloud logging read 'resource.type="cloud_run_revision" AND severity>=ERROR AND timestamp>="2026 \
    -11-05T09:00:00Z"' --limit=20 --format=json
```

Query fields, not text, where possible. The **log router** sends logs to buckets, **sinks** (BigQuery, Cloud Storage, Pub/Sub) and can **exclude** noisy entries to reduce cost. **Log-based metrics** count matching entries so you can alert on them. Use structured JSON logs with `severity` and a request id.

Metrics and alerts

Cloud Monitoring stores metrics for Google services; install the **Ops Agent** on VMs, and use **Managed Service for Prometheus** for Kubernetes. An **alerting policy** has a condition, a duration and **notification channels** (email, chat, paging). Add **uptime checks** for external probes.

Traces and errors

Cloud Trace shows where a request spent time across services; **Error Reporting** groups exceptions; **Profiler** finds hot code.

SLOs

Define a **service level indicator** (good requests divided by all requests), an **objective** (99.9 percent over 30 days) and the **error budget** (0.1 percent). Alert on **burn rate**: a fast burn (budget gone in hours) pages someone; a slow burn creates a ticket.

Diagram: From signals to action

Part	What it does	What goes wrong
Services	Emit logs, metrics and traces.	Unstructured logs are hard to query.
Logging	Stores, routes and filters log entries.	Unfiltered debug logs raise cost.
Monitoring	Metrics, dashboards and alerting policies.	CPU-only alerts miss user pain.
SLO and burn rate	Compares good requests with the objective.	A target nobody agreed on drives bad decisions.
On-call	Gets paged for fast burn only.	Alert noise causes ignored pages.

Flow: Services to Logging / Services to Monitoring / Monitoring to SLO and burn rate / SLO and burn rate to On-call

Incident: High CPU, no pager

Incident: *A slowdown went unnoticed because the alerts watched CPU while latency tripled.*

```
monitoring: run.googleapis.com/request_latencies p95 = 2.4s
monitoring: cpu utilisation = 41%
alerting policies: cpu > 80% (not firing)
```

MARGA by EknathaLabs marga.eknathalabs.com

The alert was about a cause, not the user's experience. A latency SLO with burn-rate alerts replaced the CPU page, and CPU remained on the dashboard for diagnosis.

Common mistakes

- Alerting on causes instead of symptoms
- Logging everything at debug in production
- No owner for alert policies
- SLOs without an agreed error budget policy

Check yourself

What does a log-based metric do?

Answer: Counts log entries that match a filter

What is an error budget?

Answer: Allowed unreliability under the objective

What does fast burn mean?

Answer: The budget is being used up quickly

Which tool shows time across services?

Answer: Cloud Trace

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Terraform on Google Cloud

12 minutes. Manage Google Cloud with Terraform: the google provider, remote state in Cloud Storage, service accounts for pipelines, and project factories.

Terraform is the most common way to manage Google Cloud as code. The provider is simple; the work is in state, identity and structure, which decide whether it stays safe as the team grows.

What you will learn

- Provider, backend and authentication
- Remote state in Cloud Storage
- Pipelines with federated identity
- Structure: environments, modules and project factories

Provider and state

```
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 6.0" }
  }
  backend "gcs" {
    bucket = "acme-tfstate-prod"
    prefix = "network"
  }
}

provider "google" {
  project = var.project_id
  region  = "europe-west1"
}

resource "google_compute_network" "vpc" {
  name                    = "prod-vpc"
  auto_create_subnetworks = false
}
```

The `gcs` backend stores state in a bucket with locking. Turn on **object versioning** so you can recover a bad state, restrict access to the bucket, and use **one state per layer** (network, data, apps) to limit blast radius.

Authentication

Locally use `gcloud auth application-default login`, or better `GOOGLE_IMPERSONATE_SERVICE_ACCOUNT=tf-deployer@...`. In CI use **Workload Identity Federation** so GitHub or GitLab exchanges its token for the service account with no key stored.

Structure

Use modules for patterns (a standard project, a VPC), a **project factory** module to create projects with budgets, APIs and service accounts, and review `terraform plan` output on pull requests. **Config Connector** manages Google resources from Kubernetes objects, and **Infrastructure Manager** runs Terraform as a Google-managed service.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Terraform pipeline on Google Cloud

Part	What it does	What goes wrong
Pull request	Plan output reviewed by people.	An unreviewed plan can destroy resources.
CI job	Runs fmt, validate, plan and apply.	Long-lived keys in settings can leak.
Workload Identity Federation	Token exchange for a narrow service account.	A broad role defeats the benefit.
State bucket	Versioned and locked remote state.	Public or shared access exposes secrets in state.
Google Cloud resources	The projects, networks and services managed.	Console edits cause drift.

Flow: Pull request to CI job / CI job to Workload Identity Federation / Workload Identity Federation to State bucket / Workload Identity Federation to Google Cloud resources

Incident: The leaked key

Incident: *A service account key in pipeline settings was exposed with the repository.*

```
$ gcloud iam service-accounts keys list --iam-account=tf-deployer@acme.iam.gserviceaccount.com
KEY_ID  CREATED_AT  2024-06-12
audit log: tf-deployer from 203.0.113.9 (unknown)
```

The key had wide rights and no expiry. The team deleted it, moved the pipeline to federation, narrowed the role per layer and enabled an organization policy blocking new key creation.

Common mistakes

- Committing keys or state to Git
- One giant state for everything
- A single owner-level deployer
- A state bucket without versioning or access limits

Check yourself

Where should state live on Google Cloud?

Answer: A versioned Cloud Storage bucket

What replaces key files in CI?

Answer: Workload Identity Federation

Why split state per layer?

Answer: Limit blast radius

What does a project factory do?

Answer: Creates standard projects with settings

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Security, cost and debugging on Google Cloud

14 minutes. Apply Google Cloud security controls such as IAM, organization policy and VPC Service Controls, control cost, and debug permission and quota errors.

Google Cloud is secure by design but easy to misconfigure. The same short list of controls shows up in nearly every well-run organisation, and a repeatable debugging routine saves hours.

What you will learn

- IAM least privilege and recommender
- Organization policies and VPC Service Controls
- Audit logs and cost control
- A routine for permission, quota and API errors

IAM

Grant **predefined roles** to groups at the lowest level (bucket or project, not organization). Avoid basic roles Owner, Editor and Viewer in production. Use **IAM Recommender** to remove unused permissions and **conditions** for time-limited access. Service accounts get one job each.

Organization policies and perimeters

Constraints enforced across the tree: disable service account key creation, require OS Login, restrict public IPs, restrict resource locations, enforce public access prevention. **VPC Service Controls** draw a perimeter around sensitive services (BigQuery, Cloud Storage) so data cannot be copied out even with valid credentials from outside the perimeter. **Cloud Audit Logs** record admin activity (always on) and data access (enable for sensitive data). **Security Command Center** surfaces misconfigurations.

Cost

Set budgets and alerts, export billing to BigQuery, label resources, rightsize with recommendations, delete idle disks and IPs, use committed use discounts for steady load and Spot for tolerant jobs.

Debugging errors

Error	Meaning	First step
`PERMISSION_DENIED`	Identity lacks a permission	Policy Troubleshooter for the exact permission
`API has not been used`	Service not enabled in the project	`gcloud services enable`
`RESOURCE_EXHAUSTED` / quota	Limit reached	Check quotas page; request an increase or change region
`Request is prohibited by organization's policy`	Org policy denies	View the constraint and use the approved path
VPC-SC violation	Perimeter blocked the call	Read the audit log violation entry

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Debugging a denied request

Part	What it does	What goes wrong
Error message	Names the code and often the permission.	Skimming the message hides the real cause.
Which kind	Permission, API, quota, policy or perimeter.	A wrong guess leads to a wrong fix.
Policy Troubleshooter	Shows which role would allow the call.	Granting Owner hides the real gap.
Audit logs	Show the caller, method and denial reason.	Disabled data-access logs hide reads.
Smallest fix	Grant a narrow role or enable the API.	Broad grants widen the blast radius.

Flow: Error message to Which kind / Which kind to Policy Troubleshooter / Which kind to Audit logs / Policy Troubleshooter to Smallest fix

Incident: Owner to make it work

Incident: *A permission error was silenced by granting Owner to a developer.*

```
$ gcloud projects get-iam-policy acme-pay-prod --flatten=bindings --filter=bindings.role:roles/own \
    er
user:dev@acme.example  roles/owner
troubleshooter: missing storage.objects.get on bucket acme-exports
```

The developer needed one read permission on one bucket. The team removed Owner, granted the narrow bucket role and enabled IAM Recommender reviews every quarter.

Common mistakes

- Granting Owner or Editor to fix errors
- Service account keys left in the wild
- No budgets or labels
- No data-access audit logging for sensitive data

Check yourself

What shows the exact missing permission?

Answer: Policy Troubleshooter

What do VPC Service Controls protect against?

Answer: Data leaving a perimeter

First fix for API has not been used?

Answer: Enable the API in the project

Which should you avoid in production?

Answer: Basic Owner and Editor roles

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Google Cloud

gcloud commands for projects, networks, compute, storage, Cloud Run, GKE and logs, plus a debugging order for permission errors.

Configuration

```
gcloud auth login
gcloud config configurations list
gcloud config set project acme-pay-dev
gcloud services enable compute.googleapis.com run.googleapis.com
gcloud projects get-iam-policy acme-pay-dev --format=json
```

Network and compute

```
gcloud compute networks create prod-vpc --subnet-mode=custom
gcloud compute firewall-rules list --filter="network:prod-vpc"
gcloud compute instances list --filter="labels.env=prod"
gcloud compute ssh web-1 --zone europe-west1-b --tunnel-through-iap
gcloud compute instance-groups managed list-instances web --region europe-west1
```

Storage and data

```
gcloud storage ls --long gs://acme-exports/
gcloud storage buckets describe gs://acme-exports --format='value(iamConfiguration)'
gcloud sql instances list
bq query --dry_run --use_legacy_sql=false 'select count(*) from logs.events where dt = "2026-11-05 \
    "'
```

Containers

```
gcloud run deploy web --image=REGION-docker.pkg.dev/PROJECT/app/web:1.0 --region=europe-west1 --no \
    -allow-unauthenticated
gcloud run services describe web --region europe-west1
gcloud container clusters get-credentials prod --region europe-west1
kubectl get pods -A
```

Logs

```
gcloud logging read 'resource.type="cloud_run_revision" AND severity>=ERROR' --limit=20 --freshnes \
    s=1h
gcloud pubsub subscriptions describe billing
```

Common errors

Message	First step
PERMISSION_DENIED	Policy Troubleshooter, then a narrow role
API has not been used	Enable the API in the project
Quota exceeded	Check quotas, request an increase or change region
Blocked by organization policy	Read the constraint, use the approved path
VPC Service Controls violation	Read the audit log violation entry

Safe habits

```
gcloud config list

gcloud projects get-iam-policy my-project --format=json

gcloud compute instances list --filter="status=RUNNING"

gcloud logging read 'severity>=ERROR' --limit=10 --freshness=1h

gcloud run services describe web --region=europe-west1
```

Warning: *Check* `gcloud config list` *before every change. The active project decides where a command runs.*

Danger: *Granting* `roles/owner` *or* `roles/editor` *to a service account gives it broad power. Use the narrowest predefined role.*

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Making a bucket public with* `allUsers` *exposes every object. Use uniform access and signed URLs instead.*

Warning: *Deleting a project starts a 30 day shutdown for everything in it. Confirm the project ID first.*

MARGA by EknathaLabs
marga.eknathalabs.com